Space Complexity of Recursive List Summing

RECURSIVE-LIST-SUM(float array *list*, integer length *n*)

```
if n = 0
    return 0
else
    return list[n] + RECURSIVE-LIST-SUM(list, n − 1)
end if
```

terminating condition

last

recursive formula

first (n-1)

- array *list*: size of pointer, often 8
- integer *n*: often 4
- float return place: 4

16

No

10

list[9]

9

list[8]

8 . . . .

only 16, better than previous one? 16n

Tools Options Help
Sans 12

Time Complexity of Matrix Addition

MATRIX-ADD
(integer matrix a, b, result integer matrix c, integer $rows$, $cols$)

```
for i ← 0 to rows − 1 do
    for j ← 0 to cols − 1 do
        c[i][j] ← a[i][j] + b[i][j]
    end for
end for
```

- inner for: $R = P \cdot cols + Q$
- total: $(S + R) \cdot rows + T$

total time needed: $P \cdot rows \cdot cols + (Q + S) \cdot rows + T$

Layer: Layer 1

Time Complexity of Matrix Addition

MATRIX-ADD
(integer matrix a, b, result integer matrix c, integer $rows$, $cols$)

```
for i ← 0 to rows − 1 do
    for j ← 0 to cols − 1 do
        c[i][j] ← a[i][j] + b[i][j]
    end for
end for
```

for(j=0; j<col; j++)

- inner for: $R = P \cdot cols + Q$
- total: $(S + R) \cdot rows + T$

total time needed: $P \cdot rows \cdot cols + (Q + S) \cdot rows + T$

Tools Options Help
Sans 12

Rough Time Complexity of Matrix Addition

$$P \cdot rows \cdot cols + (Q + S) \cdot rows + T$$

P, Q, R, S, T hard to keep track and not matter much

MATRIX-ADD
(integer matrix a, b, result integer matrix c, integer $rows$, $cols$)

```
for i ← 0 to rows − 1 do
    for j ← 0 to cols − 1 do
        c[i][j] ← a[i][j] + b[i][j]
    end for
end for
```

- inner for: $R = P \cdot cols + Q = \text{rough}(cols)$
- total: $(S + R) \cdot rows + T = \text{rough}(\text{rough}(cols) \cdot rows)$

rough time needed: $\text{rough}(rows \cdot cols)$

Layer: Layer 1

Tools Options Help
Sans 12

Rough Time Complexity of Matrix Addition

$$P \cdot rows \cdot cols + (Q + S) \cdot rows + T$$

P, Q, R, S, T hard to keep track and not matter much

MATRIX-ADD
(integer matrix a, b, result integer matrix c, integer $rows$, $cols$)

```
for i ← 0 to rows − 1 do
  for j ← 0 to cols − 1 do
    c[i][j] ← a[i][j] + b[i][j]
  end for
end for
```

- inner for: $R = \cancel{P} \cdot cols + \cancel{Q} = \text{rough}(cols)$
- total: $(S + R) \cdot rows + T = \text{rough}(\text{rough}(cols) \cdot rows)$

#elements of matrix

rough time needed: $\text{rough}(rows \cdot cols)$

Layer: Layer 1

Tools Options Help

Sans 12

Asymptotic Notation

Layer: Layer 1

Tools Options Help

Sans 12

Analysis Tools for Data Structures and Algorithms

Hsuan-Tien Lin

Dept. of CSIE, NTU

March 24, 2020

Layer: Layer 1

Tools Options Help

Sans 12

Motivation

ayer: Layer 1

Tools Options Help
Sans 12

Properties of Good Programs

- meet requirements, correctness: basic
- clear usage document (external), readability (internal), etc.

Resource Usage (Performance)

- efficient use of computation resources (CPU, FPU, GPU, etc.)?
  time complexity
- efficient use of storage resources (memory, disk, etc.)?
  space complexity

need: "language" for describing the complexity

Layer: Layer 1

Tools Options Help
Sans 12

Space Complexity of List Summing

LIST-SUM(float array *list*, integer length *n*)

```
total ← 0
for i ← 0 to n − 1 do
    total ← total + list[i]
end for
return total
```

- array *list*: size of pointer, often 8
- integer n: often 4
- float *total*: 4
- integer i: commonly 4
- float return place: 4

total space 24 (constant) *within algorithm execution*
does not depend on n

Layer: Layer 1

Space Complexity of List Summing

LIST-SUM(float array *list*, integer length *n*)

```
total ← 0
for i ← 0 to n − 1 do
    total ← total + list[i]
end for
return total
```

- array *list*: size of pointer, often 8
- integer *n*: often 4
- float *total*: 4
- integer *i*: commonly 4
- float return place: 4

total space 24 (constant) *within algorithm execution* does not depend on *n*

Space Complexity of Recursive List Summing

RECURSIVE-LIST-SUM(float array *list*, integer length *n*)

```
if n = 0
   return 0
else
   return list[n] + RECURSIVE-LIST-SUM(list, n − 1)
end if
```

- array *list*: size of pointer, often 8
- integer *n*: often 4
- float return place: 4

only 16, better than previous one?

Space Complexity of Recursive List Summing

RECURSIVE-LIST-SUM(float array *list*, integer length *n*)

```
if n = 0
  return 0
else
  return list[n] + RECURSIVE-LIST-SUM(list, n − 1)
end if
```

terminating condition

last

recursive formula

first (n-1)

- array *list*: size of pointer, often 8
- integer *n*: often 4
- float return place: 4

only 16, better than previous one?